

Day 29: 最終調整・動作確認をしよう

今日のゴール

アプリ全体の動作を確認し、細かい調整を行います。パフォーマンスの改善とUIの最終調整をします。

【スクリーンショット: 完成したアプリ】

なぜこれをやるのか?

リリース前の最終チェックです。**最終調整は料理の盛り付けのようなもの**。料理は完成していても、盛り付けが雑だと美味しそうに見えません。同様に、機能は完成していても、細部を整えることで、より使いやすいアプリになります。

実装ステップ一覧

ステップ	作業内容	所要時間
Step 1	全機能の動作確認	20分
Step 2	パフォーマンス改善	15分
Step 3	UI/UX の調整	15分
Step 4	ビルドとエラー確認	10分

合計時間: 約60分

Step 1: 全機能の動作確認（20分）

チェックリスト:

機能	確認項目
認証	ログイン、ログアウト、ユーザー登録
プロジェクト	作成、一覧、編集、削除、メンバー追加
タスク	作成、一覧、編集、削除、ステータス変更
タイマー	開始、停止、時間記録
コメント	投稿、編集、削除
検索	キーワード検索、フィルタ
ダッシュボード	統計、グラフ表示
レポート	週次レポート
ユーザー管理	一覧、ロール変更（管理者のみ）
プロフィール	編集、パスワード変更

動作確認の手順:

1. 新規ユーザー登録
2. ログイン
3. プロジェクト作成
4. タスク作成
5. タスクにコメント投稿
6. タイマー開始→停止
7. ダッシュボードで統計確認
8. 検索でタスクを探す
9. プロフィール編集
10. ログアウト

確認ポイント: すべての機能が正常に動作する

Step 2: パフォーマンス改善（15分）

✂ 画像の最適化:

```
// filepath: next.config.mjs
const config = {
  images: {
    domains: ['your-image-domain.com'],
    formats: ['image/avif', 'image/webp'], // 最新フォーマット
  },
};
```

✂ React.memoの使用:

```
// filepath: src/components/task/TaskCard.tsx
import { memo } from 'react';

export const TaskCard = memo(function TaskCard({ task }:
  return (
    <Card>
      { /* コンポーネント内容 */ }
    </Card>
  );
});
```

動的インポート:

```
// filepath: src/app/dashboard/page.tsx
import dynamic from 'next/dynamic';

// グラフは初回表示時だけ読み込む
const TaskStatusChart = dynamic(
  () => import('@components/dashboard/TaskStatusChart').
    (mod) => mod.TaskStatusChart
),
{ loading: () => <CircularProgress /> }
);

export default function DashboardPage() {
  return (
    <Box>
      <TaskStatusChart data={data} />
    </Box>
  );
}
```

確認ポイント: ページの読み込みが速くなる

Step 3: UI/UXの調整 (15分)

ローディング状態の改善:

```
// filepath: src/components/common/LoadingScreen.tsx
import { Box, CircularProgress, Typography } from '@mui/material';

export function LoadingScreen() {
  return (
    <Box
      sx={{
        display: 'flex',
        flexDirection: 'column',
        alignItems: 'center',
        justifyContent: 'center',
        minHeight: '50vh',
        gap: 2,
      }}
    >
      <CircularProgress size={60} />
      <Typography variant="body1" color="text.secondary">
        読み込み中...
      </Typography>
    </Box>
  );
}
```

空状態の表示:

```
// filepath: src/components/task/EmptyTaskList.tsx
import { Box, Typography, Button } from '@mui/material';
import AddIcon from '@mui/icons-material/Add';
import Link from 'next/link';

export function EmptyTaskList({ projectId }: { projectId: string }): JSX.Element {
  return (
    <Box
      sx={{
        textAlign: 'center',
        py: 8,
      }}
    >
      <Typography variant="h6" color="text.secondary" sx={{
        py: 8,
      }}>
        タスクがまだありません
      </Typography>
      <Button
        variant="contained"
        startIcon={<AddIcon />}
        component={Link}
        href={`\`/projects/${projectId}/tasks/new`}
      >
        最初のタスクを作成
      </Button>
    </Box>
  );
}
```

エラーメッセージの統一:

```
// filepath: src/components/common/ErrorMessage.tsx
import { Alert, AlertTitle } from '@mui/material';

interface ErrorMessageProps {
  title?: string;
  message: string;
}

export function ErrorMessage({ title, message }: ErrorMessageProps) {
  return (
    <Alert severity="error" sx={{ mb: 2 }}>
      {title && <AlertTitle>{title}</AlertTitle>}
      {message}
    </Alert>
  );
}
```

確認ポイント: UIが洗練され、使いやすくなる

Step 4: ビルドとエラー確認（10分）

本番ビルドの実行:

```
# filepath: ターミナル(コマンドラインで実行)
# ビルド前にリンターチェック
$ npm run lint

# 型チェック
$ npm run type-check

# 本番ビルド
$ npm run build

# ビルド結果を確認
$ npm run start
```

ビルドエラーの確認:

- ✓ Compiled successfully
- ✓ Linting and checking validity of types
- ✓ Collecting page data
- ✓ Generating static pages (10/10)
- ✓ Finalizing page optimization

Route (app)	Size	First L
└ ○ /	1.2 kB	9
└ ○ /dashboard	3.5 kB	9
└ ○ /projects	2.1 kB	9
└ ...		

確認ポイント: ビルドエラーがなく、正常に起動する

学んだこと

- **全機能の動作確認:** 体系的にテストする重要性
- **React.memo:** 不要な再レンダリングを防ぐ
- **dynamic import:** 必要なときだけコンポーネントを読み込む
- **ローディング状態:** ユーザーに待ち時間を伝える
- **空状態:** データがないときのUI
- **本番ビルド:** 実際の環境での動作確認

今日のまとめ

- ☐ 全機能の動作を確認できた
- ☐ パフォーマンスを改善できた
- ☐ UI/UXを調整できた
- ☐ ビルドエラーがないことを確認できた

⚠ リリース前チェックリスト

項目	確認
すべての機能が動作する	
ビルドエラーがない	
型エラーがない	
リンターエラーがない	
テストがパスする	
環境変数が設定されている	
セキュリティ対策ができています	

次回予告

Day 30では、完成版を公開して卒業です！おめでとうございます！